

Technical Report — Assignment 09

2317022 이다영

1. 개요

CIFAR-10 데이터셋을 이용해 2층 완전연결 신경망(Two-Layer Neural Network)을 구현하고 학습시킨다.

네트워크 구조

```
input → FC(W1, b1) → ReLU → FC(W2, b2) → Softmax → 예측
```

손실 함수: Softmax Cross-Entropy Loss + L2 Regularization

2. 구현 (neural_net.py)

2.1 loss() — Forward & Backward pass

Forward pass

```
s1 = X.dot(W1) + b1      # 1번째 FC layer
p1 = np.maximum(0, s1)    # ReLU
scores = p1.dot(W2) + b2  # 2번째 FC layer
```

Loss 계산

```
exp_s = np.exp(scores)
p2 = exp_s / np.sum(exp_s, axis=1, keepdims=True) # Softmax
data_loss = np.sum(-np.log(p2[np.arange(N), y])) / N # Cross-Entropy
regularization = reg * (np.sum(W1*W1) + np.sum(W2*W2)) # L2
loss = data_loss + regularization
```

Backward pass (역전파)

```
dscores = p2
dscores[np.arange(N), y] -= 1
dscores /= N

grads['W2'] = p1.T.dot(dscores) + 2 * reg * W2
grads['b2'] = np.sum(dscores, axis=0)

dp1 = dscores.dot(W2.T)
dp1[dp1 <= 0] = 0 # ReLU gradient

grads['W1'] = X.T.dot(dp1) + 2 * reg * W1
grads['b1'] = np.sum(dp1, axis=0)
```

2.2 train() — SGD 기반 학습

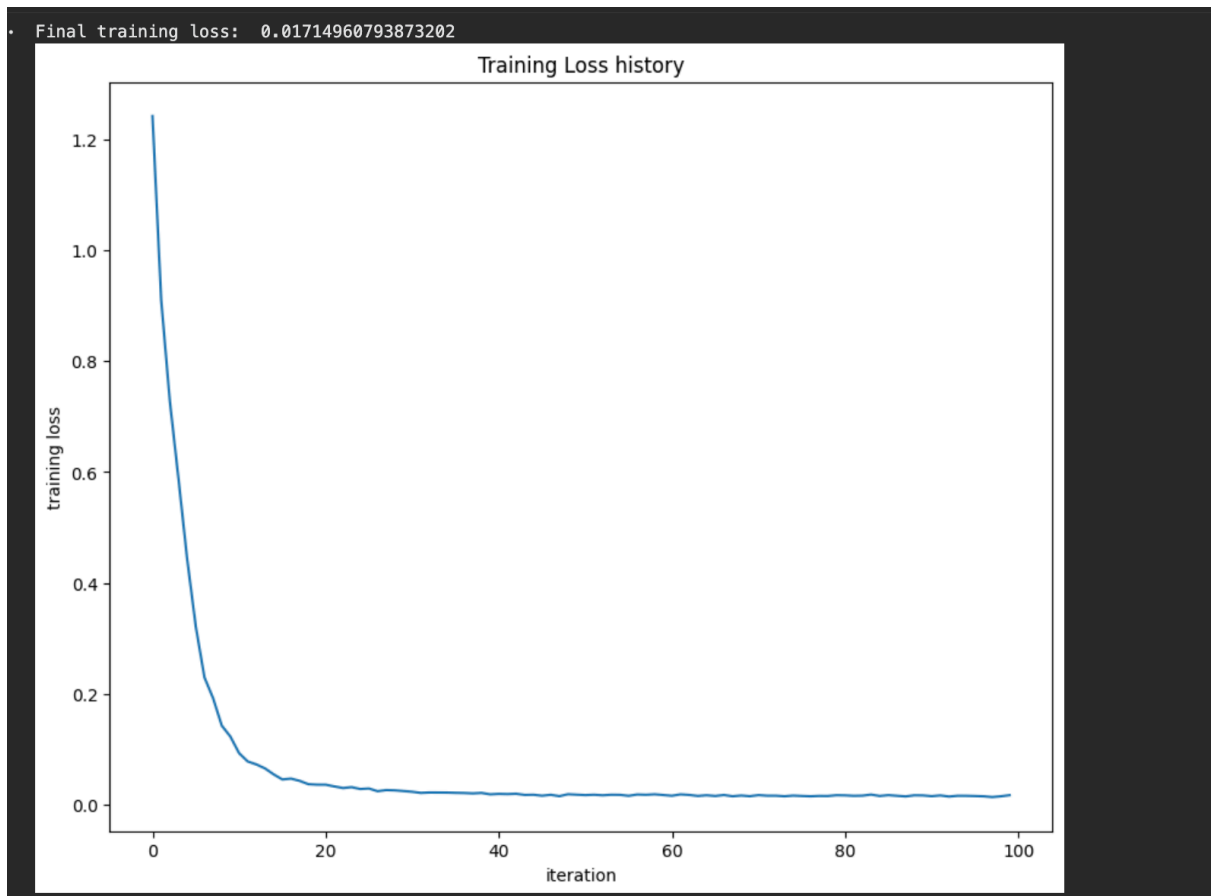
- `np.random.choice` 로 미니배치 랜덤 샘플링
- 파라미터 업데이트: `W_new = W - lr * dL/dW`
- 매 epoch마다 learning rate decay 적용 (`lr *= 0.95`)

2.3 predict() — 예측

Forward pass 수행 후 `np.argmax` 로 최대 score 클래스 반환

3. 검증 결과

항목	결과	목표
Forward pass score 오차	3.68e-08	< 1e-7
Forward pass loss 오차	1.79e-13	< 1e-12
Gradient check (W1)	3.67e-09	< 1e-8
Gradient check (W2)	3.44e-09	< 1e-8
Gradient check (b1)	2.74e-09	< 1e-8
Gradient check (b2)	4.45e-11	< 1e-8
Toy data 최종 훈련 손실	0.0171	< 0.02



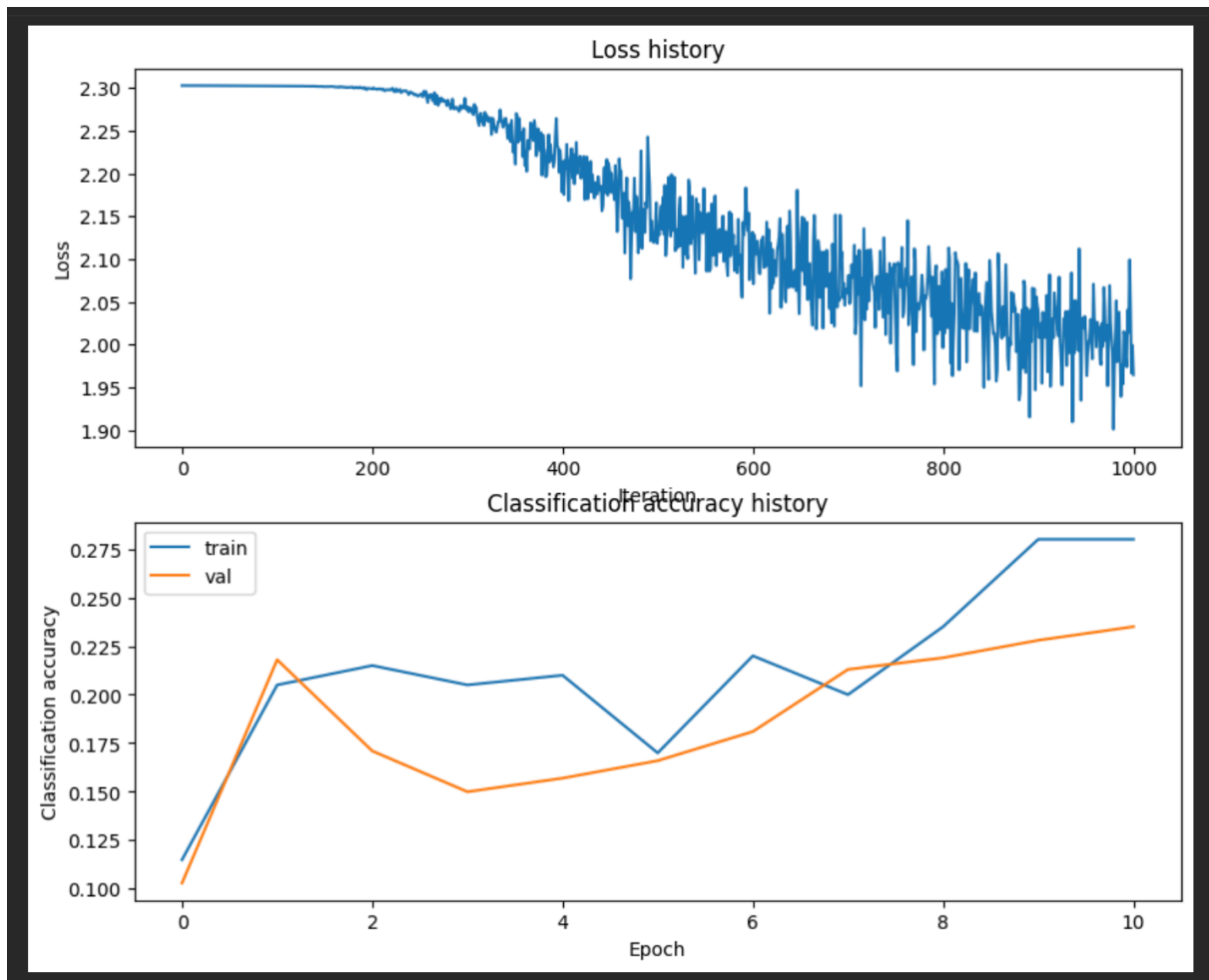
4. CIFAR-10 실험

4.1 기본 설정 학습 결과

파라미터	값
hidden_size	50
learning_rate	1e-4
reg	0.25
num_iters	1000
batch_size	200

→ **Validation accuracy: 0.241** (낮은 학습률, 작은 모델 크기가 원인)

<Loss history 그래프>



4.2 하이퍼파라미터 튜닝

기존 설정의 문제를 해결하기 위해 아래 범위로 Grid Search 수행:

파라미터	탐색 범위	기존 값
hidden_size	[100, 150]	50
learning_rate	[1e-3, 3e-3]	1e-4
reg	[0.1, 0.5]	0.25

총 $2 \times 2 \times 2 = 8$ 가지 조합 탐색, validation accuracy 기준 best_net 선정

전체 결과:

Hidden	LR	Reg	Val Acc
100	0.001	0.1	0.4700 ← best
100	0.001	0.5	0.4390

Hidden	LR	Reg	Val Acc
100	0.003	0.1	0.4580
100	0.003	0.5	0.4610
150	0.001	0.1	0.4470
150	0.001	0.5	0.4540
150	0.003	0.1	0.4340
150	0.003	0.5	0.4460

최적 하이퍼파라미터: hidden_size=100, lr=1e-3, reg=0.1

5. 최종 성능

	정확도
Train accuracy	53.8%
Validation accuracy	47.0%
Test accuracy	47.5%
Train - Test 갭	6.3%

목표 36% 초과 달성

6. Inline Question

Q. train accuracy보다 test accuracy가 낮을 때, 이 갭을 줄이는 방법은?

Answer: 1, 3

1번 (더 많은 데이터): 데이터가 많아지면 모델이 특정 패턴에 과적합되지 않고 일반화 성능이 높아져 갭이 줄어든다.

3번 (정규화 강도 증가): 정규화는 가중치를 제한하여 overfitting을 억제한다. 단, 실험에서 reg=0.1일 때 val acc 0.470으로 가장 높았고, reg=0.5로 과도하게 높이면 0.439로 낮아졌으므로 적절한 수준의 정규화가 중요하다.

2번 (hidden unit 증가) — 오답: 아래 4가지 조건에서의 성능을 비교하면, 3가지 케이스에서 hidden=100이 hidden=150보다 높은 val accuracy를 보였다. hidden unit 증가는 모델 복잡도를 높여 overfitting이 심해져 갭이 오히려 커질 수 있으므로 정답이 아니다.

LR	Reg	Hidden=100	Hidden=150	승자
0.001	0.1	0.4700	0.4470	hidden=100

LR	Reg	Hidden=100	Hidden=150	승자
0.001	0.5	0.4390	0.4540	hidden=150
0.003	0.1	0.4580	0.4340	hidden=100
0.003	0.5	0.4610	0.4460	hidden=100

7. 비교

- `data_utils.py` 의 `scipy.misc.imread` 를 최신 환경 호환을 위해 `imageio.imread` 로 수정 (scipy 버전 호환성 문제)
- Google Colab 환경에서 실행 (T4 GPU)